

JavaScript Notes with Standard Definitions

JavaScript — Detailed Notes (Standard Definitions + Simple Language + Full Code)

Made from your handwritten notebook's index. Each topic now has: a **Standard Definition** (textbook/MDN style — good for exams/interviews), an **"In simple words"** explanation, and **full code examples**. Cross-checked with TC39, MDN, and GeeksforGeeks. Style of explanation inspired by the beginner-friendly "Chai aur Code" JavaScript series.

How to use this: Read the standard definition first (write this in exams) → read "in simple words" (this is what it actually means) → run the code yourself in browser console (F12) or Node.

1. JavaScript Introduction

Standard Definition: JavaScript is a lightweight, interpreted (or just-in-time compiled), object-based scripting language with first-class functions, used primarily to create and control dynamic website content. It conforms to the ECMAScript specification maintained by TC39/Ecma International.

In simple words: JavaScript is the language that makes a webpage "alive." HTML is the skeleton, CSS is the skin/clothes, and JS is the brain/muscles — it makes things move, react, and think.

- JS is a **lightweight, interpreted, object-based scripting language**.
- It was created in **just 10 days in 1995** by Brendan Eich at Netscape.
- **Name confusion:** JS has nothing to do with Java. It was named JavaScript purely for marketing — Java was the hot language in 1995, so linking the names helped sell it.
- The actual **rulebook/spec** that defines how JS must behave is called **ECMAScript (ES)**, maintained by a committee called **TC39** (part of Ecma International).
 - Think of ECMAScript as the "recipe," and browsers (Chrome's V8, Firefox's SpiderMonkey, Safari's JavaScriptCore) as different "kitchens" that cook the recipe.
- **TC39 proposal stages** (how a new JS feature is born):

Stage	Name	Meaning (simple)
0	Strawperson	Just an idea, thrown on the table
1	Proposal	"Yes this is a real problem, let's solve it"
2	Draft	Actual syntax written down
3	Candidate	Feature is basically final, browsers start testing it
4	Finished	Officially added to JavaScript, ships to everyone

- **Big JS versions to know:**
 - **ES5 (2009)** – old-style JS (`var` , plain functions)
 - **ES6 / ES2015** – the biggest update ever: `let` , `const` , arrow functions, classes, promises, template literals
 - **ES2016 onward** – yearly small updates (async/await in ES2017, optional chaining in ES2020, private class fields in ES2022, etc.)

- **Where does JS run?**

- Inside the browser (client-side) — this is where it started.
- Outside the browser too, using **Node.js** — a runtime that lets JS run on your computer/server.

- **3 ways to add JS to a webpage:**

```
<!-- 1. Inline -->
<button onClick="alert('Hello')">Click Me</button>

<!-- 2. Internal -->
<script>
  console.log("Hello from internal script");
</script>

<!-- 3. External (best practice – keeps HTML and JS separate) -->
<script src="app.js"></script>
```

- **Why external is best:** cleaner code, reusable across pages, browser can cache the file.

2. JavaScript Basics

2.1 Comments

Standard Definition: A comment is a piece of text in the source code that is ignored by the JavaScript engine at execution time; it exists purely to document the code for human readers.

In simple words: Comments are notes you leave for yourself or your teammates. JS engine completely ignores them.

```
// This is a single line comment
console.log("Hello"); // explains what this line does

/*
  This is a
  multi-line comment
  useful for explaining a whole block
*/
```

2.2 Variables

Standard Definition: A variable is a named storage location in memory that holds a value, which can be referenced and manipulated by using its identifier (name). JavaScript provides three keywords for variable declaration: **var**, **let**, and **const**.

In simple words: A variable is like a labeled box where you keep a value so you can use it later.

```
var oldWay = "I am old style";
let modernWay = "I can change";
const fixedValue = "I cannot change";

modernWay = "Changed!"; // ✔ works
fixedValue = "Try";     // ✖ TypeError: Assignment to constant variable
```

Detailed comparison:

Feature	var	let	const
---------	-----	-----	-------

Scope	Function-scoped	Block-scoped <code>{ }</code>	Block-scoped <code>{ }</code>
Re-declare in same scope	✓ Allowed	✗ Error	✗ Error
Re-assign value	✓ Allowed	✓ Allowed	✗ Error
Hoisting	Hoisted + set to <code>undefined</code>	Hoisted but in “TDZ”	Hoisted but in “TDZ”
Best practice	Avoid (legacy)	Use when value changes	Use by default

```
// Block scope demo – the real reason let/const are better than var
if (true) {
  var a = 10;
  let b = 20;
}
console.log(a); // 10 (var leaked outside the block!)
console.log(b); // ReferenceError: b is not defined (let stayed inside block)
```

Rule of thumb: use `const` by default, switch to `let` only when the value will change, and avoid `var` in new code.

2.3 Global Variables

Standard Definition: A global variable is a variable declared outside of any function or block, having global scope — meaning it is accessible from any part of the program, including inside functions.

In simple words: A variable declared outside every function — visible everywhere in your file (and, for `var`, becomes part of the global `window` object in browsers).

```
var siteName = "MyBlog"; // global

function showName() {
  console.log(siteName); // accessible here too
}
showName(); // MyBlog
console.log(window.siteName); // "MyBlog" (only for var, not let/const)
```

- Too many globals = “global pollution” → name clashes, bugs that are hard to trace. Keep variables as local as possible.

2.4 Data Types

Standard Definition: A data type classifies the kind of value a variable can hold. ECMAScript defines seven primitive data types (String, Number, BigInt, Boolean, Undefined, Null, Symbol) and one non-primitive/reference type (Object).

In simple words: Data type = what “kind” of value is stored — text, number, true/false, etc.

7 Primitive types (stored directly, copied by value):

```
let name = "Amit";           // String
let age = 25;                 // Number
let bigNum = 123456789123456789n; // BigInt (note the 'n')
let isStudent = true;        // Boolean
let job;                     // undefined (declared, not assigned)
let noValue = null;          // null (intentionally empty)
let id = Symbol("id");       // Symbol (always unique)
```

1 Non-primitive (reference) type:

```
let person = { name: "Amit", age: 25 }; // Object (also covers arrays, functions, dates)
```

- Check type: `typeof "hi" → "string"`, `typeof 5 → "number"`
- **Famous gotcha:** `typeof null` returns `"object"` — an old bug from 1995 kept forever for backward compatibility (documented on MDN).
- **Primitive vs Reference — very important:**

```
let x = 10;
let y = x;
y = 20;
console.log(x); // 10 (unaffected – primitives are copied by value)

let obj1 = { val: 10 };
let obj2 = obj1;
obj2.val = 20;
console.log(obj1.val); // 20 (both point to the SAME object!)
```

2.5 Operators

Standard Definition: An operator is a special symbol or keyword used to perform operations on operands (values/variables) and produce a result. JavaScript operators are categorized as arithmetic, assignment, comparison, logical, bitwise, ternary, and others.

```
// Arithmetic
console.log(5 + 2, 5 - 2, 5 * 2, 5 / 2, 5 % 2, 5 ** 2);
// 7 3 10 2.5 1 25

// Comparison – the most important gotcha in JS
console.log(5 == "5"); // true (only checks value, converts type)
console.log(5 === "5"); // false (checks value AND type – always prefer this)

// Logical
console.log(true && false); // false
console.log(true || false); // true
console.log(!true); // false

// Nullish coalescing (ES2020) – different from ||
let score = 0;
console.log(score || 10); // 10 (0 is "falsy" so || ignores it – often a bug!)
console.log(score ?? 10); // 0 (?? only falls back for null/undefined, not 0)

// Optional chaining (ES2020) – avoid "cannot read property of undefined" errors
let user = { profile: { city: "Nagpur" } };
console.log(user?.profile?.city); // "Nagpur"
console.log(user?.address?.city); // undefined (no crash!)
```

2.6 If Statements

Standard Definition: The `if` statement executes a block of code only if a specified condition evaluates to `true`; `if...else` provides an alternate block when the condition is `false`.

```
let age = 20;
if (age >= 18) {
  console.log("You can vote");
}
```

```

    } else if (age >= 13) {
      console.log("You are a teenager");
    } else {
      console.log("You are a child");
    }
  }
}

```

2.7 Switch Statement

Standard Definition: The `switch` statement evaluates an expression, matches its value against a series of `case` clauses, and executes the statements associated with the first matching case; a `default` clause runs if no case matches.

In simple words: Use `switch` when you're comparing ONE variable against many fixed values — cleaner than a long if-else chain.

```

let day = 3;
switch (day) {
  case 1:
    console.log("Monday");
    break;
  case 2:
    console.log("Tuesday");
    break;
  case 3:
    console.log("Wednesday");
    break;
  default:
    console.log("Some other day");
}
// Output: Wednesday

```

⚠ Forgetting `break` causes **fall-through** — execution continues into the next case even if it doesn't match.

2.8 Loops

Standard Definition: A loop is a control structure that repeatedly executes a block of code as long as a specified condition holds true. JavaScript provides `for`, `while`, `do...while`, `for...in`, and `for...of` loops.

```

// for loop – when you know how many times to repeat
for (let i = 1; i <= 5; i++) {
  console.log(i);
}

// while loop – repeat while condition is true
let i = 1;
while (i <= 5) {
  console.log(i);
  i++;
}

// do...while – runs at least once, even if condition is false
let j = 10;
do {
  console.log(j); // prints once even though 10 <= 5 is false
} while (j <= 5);

```

```
// for...in – loops over KEYS of an object
const car = { brand: "Tata", model: "Nexon" };
for (let key in car) {
  console.log(key, car[key]); // brand Tata , model Nexon
}

// for...of – loops over VALUES of an iterable (array, string, etc.)
const fruits = ["apple", "mango", "banana"];
for (let fruit of fruits) {
  console.log(fruit);
}
```

- **break** → exit the loop completely.
- **continue** → skip current round, move to the next.

2.9 Functions

Standard Definition: A function is a reusable, self-contained block of code designed to perform a particular task, which is executed when it is invoked (called). It may optionally accept input (parameters) and return an output value.

In simple words: A function is a reusable “machine” — you put input in, it gives output back.

```
// Function declaration
function add(a, b) {
  return a + b;
}
console.log(add(2, 3)); // 5

// Function expression
const multiply = function (a, b) {
  return a * b;
};

// Arrow function (ES6) – shorter syntax
const subtract = (a, b) => a - b;

// Default parameters
function greet(name = "Guest") {
  console.log(`Hello, ${name}`);
}
greet(); // Hello, Guest
greet("Priya"); // Hello, Priya

// Rest parameters – collect unlimited args into an array
function sumAll(...numbers) {
  return numbers.reduce((total, n) => total + n, 0);
}
console.log(sumAll(1, 2, 3, 4)); // 10
```

- **Arrow functions don't have their own `this`** — they borrow `this` from where they were written (explained in detail in section 11.1).

3. JavaScript Objects (Built-in)

3.1 Object

Standard Definition: An object is a non-primitive data type consisting of a collection of properties, where each property is defined as a key-value pair; keys are strings (or Symbols) and values can be any data type, including functions (called methods).

In simple words: An object is a real-world “thing” described using properties — like describing a person with name, age, city.

```
const student = {
  name: "Riya",
  age: 21,
  isPassed: true,
  greet: function () {
    console.log(`Hi, I'm ${this.name}`);
  },
};

console.log(student.name);      // dot notation
console.log(student["age"]);    // bracket notation (useful for dynamic keys)
student.greet();               // Hi, I'm Riya

// Adding / deleting properties dynamically
student.city = "Nagpur";
delete student.isPassed;
console.log(student);

// Useful object methods
console.log(Object.keys(student)); // ["name", "age", "greet", "city"]
console.log(Object.values(student));
console.log(Object.entries(student)); // [ [key,value], [key,value] ...]
```

3.2 Array

Standard Definition: An array is a special type of object used to store an ordered, index-based collection of multiple values (of any data type) under a single variable name.

In simple words: An ordered list of items, like a train with numbered compartments starting from 0.

```
const numbers = [10, 20, 30, 40];

numbers.push(50);      // add to end      → [10,20,30,40,50]
numbers.pop();         // remove from end → [10,20,30,40]
numbers.unshift(5);    // add to start   → [5,10,20,30,40]
numbers.shift();       // remove from start → [10,20,30,40]

console.log(numbers.length); // 4

// The "big 3" array methods every JS dev must know:
const doubled = numbers.map(n => n * 2);      // [20,40,60,80]
const big = numbers.filter(n => n > 15);      // [20,30,40]
const total = numbers.reduce((sum, n) => sum + n, 0); // 100

numbers.forEach(n => console.log(n));          // just loops, no return value

console.log(numbers.includes(20)); // true
console.log(numbers.indexOf(30));  // 2
console.log(numbers.slice(1, 3));  // [20,30] (doesn't change original)
numbers.splice(1, 1, 99);          // removes 1 item at index 1, inserts 99 (changes original!)
```

3.3 String

Standard Definition: A string is a primitive data type representing a sequence of characters used to store and manipulate text; strings in JavaScript are immutable, meaning their content cannot be changed after creation.

In simple words: Text wrapped in quotes. You can't change a character directly — you always create a new string.

```
const msg = "Hello World";

console.log(msg.length);           // 11
console.log(msg.toUpperCase());    // HELLO WORLD
console.log(msg.toLowerCase());    // hello world
console.log(msg.indexOf("World")); // 6
console.log(msg.slice(0, 5));      // Hello
console.log(msg.replace("World", "JS")); // Hello JS
console.log(msg.trim());           // removes extra spaces from both ends
console.log(msg.split(" "));      // ["Hello","World"]

// Template literals (ES6) – the modern way to build strings
const name = "Amit";
const age = 25;
console.log(`My name is ${name} and I am ${age} years old.`);
// supports multi-line too:
const para = `Line 1
Line 2`;
```

3.4 Date

Standard Definition: The `Date` object is a built-in JavaScript object used to represent and work with a single moment in time, stored internally as the number of milliseconds elapsed since the Unix epoch (January 1, 1970, UTC).

```
const now = new Date();
console.log(now.getFullYear()); // e.g. 2026
console.log(now.getMonth());    // 0-indexed! January = 0
console.log(now.getDate());     // day of month
console.log(now.getDay());      // day of week, 0 = Sunday
console.log(now.getHours(), now.getMinutes(), now.getSeconds());
console.log(now.getTime());     // milliseconds since Jan 1 1970 (epoch)

// Creating a specific date
const birthday = new Date(2000, 4, 15); // May 15, 2000 (month is 0-indexed!)
```

3.5 Math

Standard Definition: `Math` is a built-in JavaScript object (not a constructor) that provides properties and static methods for performing mathematical constants and functions.

```
console.log(Math.round(4.6)); // 5
console.log(Math.floor(4.9)); // 4 (always rounds down)
console.log(Math.ceil(4.1));  // 5 (always rounds up)
console.log(Math.abs(-7));    // 7
console.log(Math.max(3, 7, 2)); // 7
console.log(Math.min(3, 7, 2)); // 2
```

```

console.log(Math.pow(2, 3));    // 8
console.log(Math.sqrt(16));    // 4

// Random number between 0 and 1
console.log(Math.random());

// Random integer between min and max (inclusive) – common interview question
function randomInt(min, max) {
    return Math.floor(Math.random() * (max - min + 1)) + min;
}
console.log(randomInt(1, 6)); // dice roll simulator

```

3.6 Number

Standard Definition: `Number` is a primitive wrapper object/data type representing numeric values, including integers and floating-point numbers, following the IEEE 754 double-precision standard.

```

console.log(Number.isInteger(5));    // true
console.log(Number.isInteger(5.5));  // false
console.log(Number("123"));          // 123 (string to number)
console.log((3.14159).toFixed(2));   // "3.14" (rounds to 2 decimals)
console.log(Number.parseFloat("3.14abc")); // 3.14
console.log(Number.MAX_SAFE_INTEGER); // 9007199254740991

```

3.7 Boolean

Standard Definition: `Boolean` is a primitive data type that can hold only one of two logical values: `true` or `false`, typically used for conditional testing.

```

console.log(Boolean(0));             // false
console.log(Boolean(""));            // false
console.log(Boolean(null));           // false
console.log(Boolean(undefined));     // false
console.log(Boolean(NaN));            // false
console.log(Boolean("0"));            // true  (non-empty string is truthy!)
console.log(Boolean([]));             // true  (empty array is truthy!)
console.log(Boolean({}));            // true  (empty object is truthy!)

```

In simple words: Only 6 values are “falsy” in JS: `0`, `""`, `null`, `undefined`, `NaN`, `false`. Everything else is truthy — this catches beginners off guard (`"0"` and `[]` being truthy).

4. JavaScript BOM (Browser Object Model)

Standard Definition: The Browser Object Model (BOM) is a browser-provided (non-standardized) set of objects that allows JavaScript to interact with the browser window itself — including navigation history, screen properties, and the URL — as opposed to the page content, which is handled by the DOM.

In simple words: BOM lets JS talk to the browser window itself — not the webpage content, but things like the URL bar, screen size, browsing history.

```

// window – the global object; everything (including document) lives inside it
console.log(window.innerWidth, window.innerHeight);

// navigator – info about the browser/device

```

```

console.log(navigator.userAgent);
console.log(navigator.language);

// screen – info about the user's display
console.log(screen.width, screen.height);

// location – current URL, and can redirect
console.log(location.href);
// location.href = "https://google.com"; // redirects the page
location.reload(); // refreshes the page

// history – session navigation
history.back(); // like clicking browser's back button
history.forward();

```

- BOM is **not part of the official ECMAScript spec** — every browser implements it a bit differently (though `window`, `navigator`, `location` are practically standard now).

5. JavaScript DOM (Document Object Model)

Standard Definition: The Document Object Model (DOM) is a programming interface (and W3C/WHATWG standard) that represents an HTML/XML document as a hierarchical tree of node objects, allowing programs like JavaScript to dynamically access, traverse, and modify the document's structure, style, and content.

In simple words: When the browser loads your HTML, it converts it into a tree-like structure of “nodes” in memory. The DOM is how JS reads and changes that tree.

```

<div id="box">Hello</div>
<p class="text">Some text</p>

```

```

// Selecting elements
const box = document.getElementById("box");
const texts = document.getElementsByClassName("text"); // returns HTMLCollection
const paragraphs = document.getElementsByTagName("p");
const first = document.querySelector(".text"); // CSS-selector style, returns FIRST match
const all = document.querySelectorAll("p"); // returns ALL matches (NodeList)

// Reading / changing content
console.log(box.innerHTML); // "Hello"
box.innerHTML = "<b>Updated!</b>"; // can insert HTML tags
box.textContent = "Plain text only"; // safer – doesn't parse HTML

// Changing styles
box.style.color = "red";
box.style.backgroundColor = "yellow";

// Working with classes (best practice over inline styles)
box.classList.add("highlight");
box.classList.remove("highlight");
box.classList.toggle("highlight"); // adds if missing, removes if present

// Attributes
box.setAttribute("data-info", "test");
console.log(box.getAttribute("data-info"));

// Creating and adding new elements

```

```
const newPara = document.createElement("p");
newPara.textContent = "I'm new here!";
document.body.appendChild(newPara);

// Removing elements
newPara.remove();
```

6. JavaScript Validation

Standard Definition: Form validation is the process of checking user-supplied input against a set of rules (required fields, format, length, range) before it is accepted or submitted, in order to ensure the data is correct and complete.

In simple words: Before sending a form to the server, check that the user actually filled it correctly — catch mistakes early and give instant feedback.

```
<form id="signupForm">
  <input type="text" id="username" />
  <input type="email" id="email" />
  <input type="password" id="password" />
  <button type="submit">Sign Up</button>
</form>
```

```
document.getElementById("signupForm").addEventListener("submit", function (e) {
  e.preventDefault(); // stop the form from actually submitting

  const username = document.getElementById("username").value.trim();
  const email = document.getElementById("email").value.trim();
  const password = document.getElementById("password").value;

  if (username === "") {
    alert("Username is required");
    return;
  }

  const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  if (!emailPattern.test(email)) {
    alert("Enter a valid email");
    return;
  }

  if (password.length < 6) {
    alert("Password must be at least 6 characters");
    return;
  }

  alert("Form is valid! Submitting...");
  // this.submit();
});
```

⚠ **Important:** Client-side validation is only for user experience. A hacker can bypass JS entirely, so the **server must always re-validate** everything. Both MDN and GfG stress this as a security basic.

7. JavaScript OOPs (Object-Oriented Programming)

Standard Definition: Object-Oriented Programming (OOP) is a programming paradigm based on the concept of “objects,” which bundle data (properties) and behavior (methods) together. Its four core pillars are Encapsulation, Inheritance, Polymorphism, and Abstraction.

In simple words: OOP is about modeling real-world things as objects that hold their own data and behavior.

7.1 Class

Standard Definition: A class is a blueprint or template (introduced in ES6 as syntactic sugar over JavaScript's existing prototype-based inheritance) used to create objects with predefined properties and methods.

```
class Person {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }
  greet() {
    console.log(`Hi, I'm ${this.name}, ${this.age} years old`);
  }
}

const p1 = new Person("Riya", 22);
p1.greet(); // Hi, I'm Riya, 22 years old
```

- `class` is **syntactic sugar** — under the hood, JS classes still use the same old prototype system. Common interview Q: “Is JS truly OOP?” Answer: it’s **prototype-based**, and `class` just makes the syntax look like Java/C++.

7.2 Object

Standard Definition: In OOP terms, an object is a specific instance of a class, created using the `new` keyword, that contains actual values for the properties defined by that class.

```
const p2 = new Person("Aman", 30); // p2 is an "instance" (object) of class Person
console.log(typeof p2); // object
console.log(p2 instanceof Person); // true
```

7.3 Prototype

Standard Definition: A prototype is an internal object linked to every JavaScript object (accessible via `[[Prototype]]` / `__proto__`) from which it can inherit properties and methods; this mechanism is called prototypal inheritance.

```
function Animal(name) {
  this.name = name;
}
Animal.prototype.speak = function () {
  console.log(`${this.name} makes a sound`);
};

const dog = new Animal("Dog");
dog.speak(); // Dog makes a sound
```

```
// Under the hood, class methods are also stored on the prototype:
console.log(Person.prototype.greet); // the function itself
```

In simple words: Instead of copying the `speak` function into every object (wasting memory), JS stores it once on the `prototype`, and every object made from that constructor/class shares the same copy.

7.4 Constructor Method

Standard Definition: A constructor is a special method of a class, named `constructor`, that is automatically invoked when a new instance of the class is created using the `new` keyword; it is typically used to initialize the object's properties.

```
class Car {
  constructor(brand, model) {
    console.log("Constructor called!");
    this.brand = brand;
    this.model = model;
  }
}
const c1 = new Car("Tata", "Nexon"); // "Constructor called!" prints immediately
```

- Runs **automatically** the moment `new ClassName()` executes. Only one constructor allowed per class.

7.5 Static Method

Standard Definition: A static method is a method defined on a class itself using the `static` keyword, rather than on instances of that class; it is called directly on the class and cannot be accessed through an instance.

```
class MathHelper {
  static square(x) {
    return x * x;
  }
  static PI = 3.14159; // static property (ES2022+)
}
console.log(MathHelper.square(5)); // 25
// const m = new MathHelper();
// m.square(5); // ✗ Error – static methods belong to the CLASS, not instances
```

7.6 Encapsulation

Standard Definition: Encapsulation is the OOP principle of bundling data and the methods that operate on that data within a single unit (a class), while restricting direct external access to some of an object's internal details — typically implemented using private fields/methods.

```
class BankAccount {
  #balance = 0; // private field (ES2022) – # makes it truly inaccessible from outside

  deposit(amount) {
    if (amount > 0) this.#balance += amount;
  }
  getBalance() {
    return this.#balance;
  }
}
const acc = new BankAccount();
```

```
acc.deposit(500);
console.log(acc.getBalance()); // 500
console.log(acc.#balance);      // ✖ SyntaxError – cannot access private field from outside
```

7.7 Inheritance

Standard Definition: Inheritance is the OOP mechanism by which one class (the child/subclass) acquires the properties and methods of another class (the parent/superclass), implemented in JavaScript using the `extends` keyword and the `super()` call.

```
class Animal {
  constructor(name) {
    this.name = name;
  }
  eat() {
    console.log(`${this.name} is eating`);
  }
}

class Dog extends Animal {
  constructor(name, breed) {
    super(name); // calls the parent's constructor
    this.breed = breed;
  }
  bark() {
    console.log(`${this.name} is barking`);
  }
}

const d1 = new Dog("Tommy", "Labrador");
d1.eat(); // inherited from Animal → Tommy is eating
d1.bark(); // own method → Tommy is barking
```

7.8 Polymorphism

Standard Definition: Polymorphism (“many forms”) is the OOP principle whereby a method with the same name behaves differently depending on the object (class) that invokes it — commonly achieved through method overriding in subclasses.

```
class Shape {
  area() {
    return 0;
  }
}

class Circle extends Shape {
  constructor(radius) {
    super();
    this.radius = radius;
  }
  area() {
    return Math.PI * this.radius ** 2; // overrides parent's area()
  }
}

class Square extends Shape {
  constructor(side) {
    super();
    this.side = side;
  }
}
```

```

    }
    area() {
        return this.side ** 2; // different implementation, same method name
    }
}

const shapes = [new Circle(5), new Square(4)];
shapes.forEach(shape => console.log(shape.area().toFixed(2)));
// 78.54
// 16.00

```

7.9 Abstraction

Standard Definition: Abstraction is the OOP principle of hiding complex implementation details and exposing only the essential features/interface of an object to the outside world.

```

class Vehicle {
    constructor(type) {
        if (this.constructor === Vehicle) {
            throw new Error("Cannot instantiate an abstract class directly!");
        }
        this.type = type;
    }
    start() {
        throw new Error("start() must be implemented by subclass");
    }
}

class Bike extends Vehicle {
    start() {
        console.log("Bike started with a kick 🦵");
    }
}

// new Vehicle("generic"); // ✖ throws our custom error
new Bike().start();         // ✔ Bike started with a kick

```

In simple words: JS has no built-in `abstract` keyword (unlike Java), so developers simulate it by throwing errors if someone tries to use the “template” class directly.

8. JavaScript Cookies

Standard Definition: A cookie is a small piece of data (maximum ~4KB), stored in the user’s browser as a key-value pair, that is automatically sent to the server with every HTTP request to the domain that set it — commonly used for session management and personalization.

```

// Setting a cookie
document.cookie = "username=Amit; expires=Fri, 31 Dec 2026 23:59:59 UTC; path=/";

// Reading all cookies (returns one long string)
console.log(document.cookie); // "username=Amit; theme=dark; ..."

// Helper function to read one specific cookie
function getCookie(name) {
    const value = `; ${document.cookie}`;
    const parts = value.split(`; ${name}=`);
    if (parts.length === 2) return parts.pop().split(";").shift();
}

```

```

}
console.log(getCookie("username")); // "Amit"

// Deleting a cookie – set its expiry date in the past
document.cookie = "username=; expires=Thu, 01 Jan 1970 00:00:00 UTC; path=/";

```

- **Modern alternative (MDN-recommended for pure client storage):**

```

localStorage.setItem("theme", "dark"); // stays even after closing browser
console.log(localStorage.getItem("theme"));
sessionStorage.setItem("temp", "123"); // cleared when tab closes

```

9. JavaScript Events

Standard Definition: An event is a signal that a specific occurrence has happened in the browser (such as a user action or a state change), which JavaScript can “listen” for via an event handler and respond to accordingly.

```

<button id="myBtn">Click Me</button>

```

```

const btn = document.getElementById("myBtn");

// Best practice: addEventListener (allows multiple listeners, cleaner)
btn.addEventListener("click", function (event) {
  console.log("Button clicked!");
  console.log(event.target); // the element that triggered the event
});

// Common events
document.addEventListener("keydown", (e) => console.log("Key pressed:", e.key));
window.addEventListener("load", () => console.log("Page fully loaded"));
document.getElementById("myInput")?.addEventListener("change", (e) =>
console.log(e.target.value));

```

Event Bubbling (Standard Definition: a propagation mechanism where an event triggered on a child element travels upward through its ancestor elements):

```

<div id="parent">
  <button id="child">Click</button>
</div>

```

```

document.getElementById("parent").addEventListener("click", () => console.log("Parent clicked"));
document.getElementById("child").addEventListener("click", () => console.log("Child clicked"));
// Clicking the button logs:
// Child clicked
// Parent clicked (bubbles up!)

```

- `event.stopPropagation()` — stops the bubbling.
- `event.preventDefault()` — stops the browser’s default action.

10. Exception Handling

Standard Definition: Exception handling is the process of responding to runtime errors (exceptions) in a controlled manner using `try`, `catch`, and `finally` blocks, preventing the program from crashing abruptly. The `throw` statement is used to manually raise a custom exception.

```
function divide(a, b) {
  if (b === 0) {
    throw new Error("Cannot divide by zero!"); // manually create an error
  }
  return a / b;
}

try {
  console.log(divide(10, 0));
} catch (error) {
  console.log("Something went wrong:", error.message);
} finally {
  console.log("This always runs, error or not");
}

// Output:
// Something went wrong: Cannot divide by zero!
// This always runs, error or not
```

Common built-in error types: | Error type | When it happens | |—| | `SyntaxError` | Code is written wrong (missing bracket, etc.) | | `ReferenceError` | Using a variable that doesn't exist | | `TypeError` | Using a value in a way its type doesn't allow (e.g. calling a non-function) | | `RangeError` | A number is outside an allowed range |

```
// try/catch also works great with async/await
async function fetchData() {
  try {
    const res = await fetch("https://api.example.com/data");
    const data = await res.json();
    console.log(data);
  } catch (err) {
    console.log("Fetch failed:", err.message);
  }
}
```

11. JavaScript Misc Topics

11.1 `this` Keyword

Standard Definition: `this` is a keyword that refers to the object that is currently executing the code — its value is determined dynamically by the invocation context (how the function is called), not by where the function is defined (except for arrow functions, which use lexical `this`).

```
const user = {
  name: "Amit",
  regularFunc: function () {
    console.log(this.name); // "this" = user object → "Amit"
  },
  arrowFunc: () => {
    console.log(this.name); // arrow has NO own "this", takes it from outer scope → undefined
  },
};
```

```
user.regularFunc(); // Amit
user.arrowFunc();   // undefined
```

11.2 Debugging

Standard Definition: Debugging is the systematic process of identifying, isolating, and fixing defects (bugs) in code, commonly done using console logging, breakpoints, and browser developer tools.

```
console.log("simple value check");
console.error("something failed");
console.table([ { name: "A", age: 1 }, { name: "B", age: 2 } ]); // pretty table in console
debugger; // pauses code here IF DevTools is open – great for step-by-step inspection
```

11.3 Hoisting

Standard Definition: Hoisting is JavaScript's default behavior of moving declarations (of variables and functions) to the top of their containing scope during the compilation phase, before code execution begins.

```
console.log(a); // undefined (not an error!) – var is hoisted
var a = 5;

console.log(b); // ✖ ReferenceError: Cannot access 'b' before initialization
let b = 10;

sayHi(); // "Hi!" – works even before the definition below, because function DECLARATIONS are
fully hoisted
function sayHi() {
  console.log("Hi!");
}
```

11.4 Strict Mode

Standard Definition: Strict mode is an ECMAScript5 feature, activated with the directive `"use strict";`, that enforces a stricter set of parsing and error-handling rules on JavaScript code, eliminating some silent errors and unsafe actions.

```
"use strict";
x = 10; // ✖ ReferenceError: x is not defined (without strict mode, this would silently create a
global!)
```

- Classes and ES6 modules are **strict mode by default**.

11.5 JavaScript Promise

Standard Definition: A Promise is an object representing the eventual completion (or failure) of an asynchronous operation, existing in one of three states — pending, fulfilled, or rejected — and used to handle asynchronous results without deeply nested callbacks.

```
function orderChai() {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      const chaiReady = true;
      if (chaiReady) resolve("Chai is ready 🍵");
      else reject("Chai spilled!");
    });
  });
}
```

```

    }, 2000);
  });
}

orderChai()
  .then((msg) => console.log(msg))
  .catch((err) => console.log(err));

// Modern way – async/await (cleaner, reads top-to-bottom like normal code)
async function getChai() {
  try {
    const result = await orderChai();
    console.log(result);
  } catch (err) {
    console.log(err);
  }
}
getChai();

```

11.6 Compare Dates

Standard Definition: Date comparison in JavaScript is performed by converting `Date` objects into their numeric timestamp representation (milliseconds since epoch, via `.getTime()`) and comparing those numeric values using standard relational operators.

```

const date1 = new Date("2026-01-01");
const date2 = new Date("2026-06-15");

console.log(date1 < date2);           // true
console.log(date1.getTime() === date2.getTime()); // false

const diffMs = date2 - date1; // subtracting dates gives milliseconds difference
const diffDays = diffMs / (1000 * 60 * 60 * 24);
console.log(diffDays); // 165

```

11.7 Array `.length`

Standard Definition: `.length` is a built-in, writable property of an array object that returns (or sets) the number of elements it contains.

```

const arr = [1, 2, 3, 4, 5];
console.log(arr.length); // 5

arr.length = 3; // truncates the array!
console.log(arr); // [1, 2, 3]

```

11.8 `alert()`

Standard Definition: `alert()` is a method of the `window` object that displays a modal dialog box with a specified message and an OK button, pausing script execution until the user dismisses it.

```

alert("Welcome to the site!"); // simple popup – pauses code until user clicks OK

```

11.9 `eval()` function

Standard Definition: `eval()` is a global function that evaluates a string as JavaScript source code and executes it within the current execution context.

```
console.log(eval("2 + 2")); // 4 – runs a string AS CODE
```

⚠ **MDN strongly warns against using `eval()`** — huge security risk (can run malicious injected code) and it's slow because the JS engine can't optimize code inside it ahead of time.

11.10 `closest()`

Standard Definition: `element.closest(selector)` is a DOM method that searches the element itself and its ancestors (traversing up the DOM tree) for the nearest one that matches the specified CSS selector, returning `null` if none is found.

```
<div class="card">
  <button id="likeBtn">Like</button>
</div>
```

```
const btn = document.getElementById("likeBtn");
const card = btn.closest(".card"); // walks UP the DOM tree to find nearest ".card" ancestor
console.log(card); // <div class="card">...</div>
```

11.11 `continue` statement

Standard Definition: The `continue` statement terminates execution of the statements in the current iteration of a loop and proceeds immediately to the next iteration's condition check.

```
for (let i = 1; i <= 5; i++) {
  if (i === 3) continue; // skip 3, don't print it
  console.log(i);
}
// Output: 1 2 4 5
```

11.12 `getAttribute()` method

Standard Definition: `element.getAttribute(name)` is a DOM method that returns the value of a specified attribute on an element, or `null` if the attribute does not exist.

```

```

```
const img = document.getElementById("pic");
console.log(img.getAttribute("src")); // "cat.jpg"
console.log(img.getAttribute("alt")); // "A cute cat"
img.setAttribute("src", "dog.jpg"); // changes the image source
```

Bonus: A Few Extra Concepts Often Taught Right After This Syllabus

- **Destructuring (ES6):**

```
const { name, age } = { name: "Amit", age: 25 };
const [first, second] = [10, 20];
```

- **Spread operator (...):**

```
const arr1 = [1, 2, 3];  
const arr2 = [...arr1, 4, 5]; // [1,2,3,4,5] – copies + adds  
const objCopy = { ...student }; // shallow copy of an object
```

- **Modules (import / export)** — splitting code across files (ES6):

```
// math.js  
export function add(a, b) { return a + b; }  
// app.js  
import { add } from './math.js';
```

- **fetch() API** — making network requests from the browser:

```
fetch("https://api.example.com/users")  
  .then(res => res.json())  
  .then(data => console.log(data))  
  .catch(err => console.log(err));
```

Quick Reference — Trusted Sources Used

Topic	Reference
Official spec & proposal stages	tc39.es/process-document
Syntax, methods, browser support	developer.mozilla.org (MDN)
Practice problems, extra examples	geeksforgeeks.org/javascript
Beginner-friendly teaching style, code repos	Chai aur Code / Chai aur JavaScript by Hitesh Choudhary

Tip: In exams, write the “Standard Definition” line first — that’s the textbook-style answer examiners look for. Then use “in simple words” + code to make sure you actually understand it, not just memorized it.